



**POLITECNICO
DI TORINO**

Dipartimento di
Elettronica e
Telecomunicazioni



DIGITAL SYSTEMS ELECTRONICS

Academic year 2024-2025

LABORATORY NR. 5

DUE DATE: March 31 2025

DELIVERY DATE: April 14, 2025

GROUP 20

CONTRIBUTIONS:

- Erfan Taheri
- Raman Jahandari
- Bardia Vafadar
- Ali Mansouri

The members of the group listed above declare under their own responsibility that no part of this document has been copied from other documents and that the associated code is original and has been developed expressly for the assigned project.

Introduction

The purpose of this lab is to explore Finite State Machines (FSMs) and understand how to implement them in VHDL. In the first part, an FSM is designed to detect a 4-clock sequence of either 0s or 1s. The second part modifies this FSM to use an all zero reset state, making it more compatible with FPGA hardware. In the third part, the FSM is restructured using a two-process model, without explicitly writing out the logic expressions for each flip-flop. Finally, a circuit is implemented that scrolls the word “HELLO” across the 7-segment displays of the DE1 board, controlled by an FSM.

Part 1. One-Hot Finite state machine

The first part of this laboratory involved implementing a Finite State Machine (FSM) designed to detect a sequence of four consecutive 0s or 1s on the input signal. When such a sequence was detected, the FSM set the output signal z to 1; otherwise, z remained 0. One-hot encoding was used to define nine distinct states, with each state represented by one bit in a 9-bit vector y . Based on the state diagram, logic expressions were derived for each transition, where a state became active based on a specific combination of the previous state and the input value:

$$\begin{aligned}y(0) &= \overline{w} \cdot (y_0 + y_1 + y_2 + y_3 + y_4 + y_5 + y_6 + y_7 + y_8) \\y(1) &= w \cdot y_0 \\y(2) &= w \cdot y_1 \\y(3) &= w \cdot y_2 \\y(4) &= w \cdot y_3 \\y(5) &= \overline{w} \cdot y_0 \\y(6) &= \overline{w} \cdot y_1 \\y(7) &= \overline{w} \cdot y_2 \\y(8) &= \overline{w} \cdot y_3\end{aligned}$$

The output is given by $z = y(4) + y(8)$, which becomes high when either four consecutive 1s or four consecutive 0s are detected.

To implement the identified logic expressions, the FSM was modeled using a set of nine flip-flops, each representing a single state in the one-hot encoding scheme. The following code describes a D-type flip-flop with an asynchronous active-low reset, which ensured that all state bits were reset to 0 when the system was initialized. This flip-flop design served as the basic building block for the FSM implementation:

```
9  ARCHITECTURE Behavior OF Dflipf_flop IS
10 BEGIN
11   PROCESS (Clock, Resetn)
12   BEGIN
13     IF (Resetn = '0' and rising_edge(Clock)) THEN
14       Q <= '0';
15     ELSIF (rising_edge(Clock)) THEN
16       Q <= D;
17     END IF;
18   END PROCESS;
19 END Behavior;
20 --Flip-flop with asynchronous reset
```

The FSM is implemented using a 9-bit vector *y*, where each bit represents a unique state in the One-Hot encoding scheme. Initially, the FSM is reset to state A by setting only *y*(0) to '1'. The FSM then transitions between states based on the input signal *w*. When *w* = '1', the FSM advances from *y*(0) through *y*(4), representing the detection of four consecutive 1s. Similarly, when *w* = '0', the FSM moves from *y*(0) through *y*(8) using states *y*(5) to *y*(7). The logic for *y*(0) combines all possible transitions that return the FSM to state A, handling mismatched input cases. The output signal *z* is asserted high when either *y*(4) or *y*(8) is active, indicating a successful detection of four identical bits.

```

15 begin
16   process(cclk)
17   begin
18     if rising_edge(cclk) then
19       if reset_n = '0' then
20         y <= (0 => '1', others => '0'); -- Reset to state A
21       else
22         -- Next state logic
23         y(0) <= (not w) and (y(0) or y(1) or y(2) or y(3) or y(4) or
24           y(5) or y(6) or y(7) or y(8));
25         y(1) <= w and y(0);
26         y(2) <= w and y(1);
27         y(3) <= w and y(2);
28         y(4) <= w and y(3);
29         y(5) <= (not w) and y(0);
30         y(6) <= (not w) and y(5);
31         y(7) <= (not w) and y(6);
32         y(8) <= (not w) and y(7);
33       end if;
34     end if;
35   end process;
36
37   z <= y(4) or y(8); -- Output logic
38 end Behavioral;
39

```

The testbench applies a reset and drives the input *w* with sequences of four 0s, 1s, and 0s. A 20 ns clock is used, and the FSM output *z* is expected to go high after each valid sequence, confirming correct detection.

```

35 clk_process: process
36 begin
37   while now < 500 ns loop
38     clk <= '0'; wait for 10 ns;
39     clk <= '1'; wait for 10 ns;
40   end loop;
41   wait;
42 end process;
43
44 -- Stimulus process matching the lab waveform
45 stim_proc: process
46 begin
47   -- Initial reset
48   reset_n <= '0';
49   wait for 20 ns; -- Reset for 1 clock cycle
50   reset_n <= '1';
51
52   -- w = 0 for 4 clocks
53   w <= '0'; wait for 80 ns;
54
55   -- w = 1 for 4 clocks
56   w <= '1'; wait for 80 ns;
57
58   -- w = 0 for 4 clocks
59   w <= '0'; wait for 80 ns;
60
61   -- Continue with high or leave constant for clean ending
62   w <= '1'; wait;
63 end process;

```

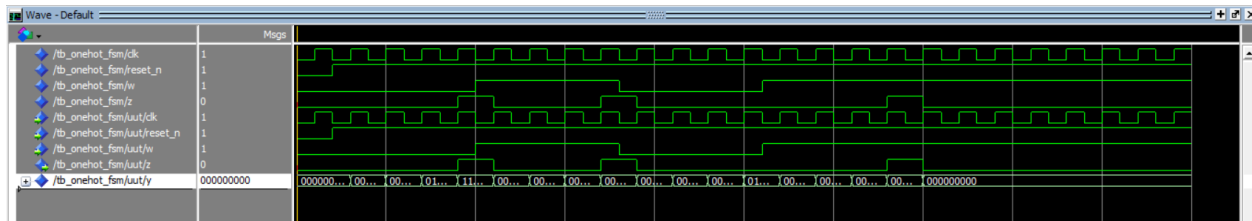


Figure 1. FSM simulation waveform showing input, output, and state transitions.

The waveform shows the FSM behavior in response to input sequences of four '0's or '1's. The output z goes high immediately after each valid sequence, confirming that the detection logic works as intended. The internal state vector y also changes correctly, progressing through the one-hot encoded states. As shown in Figure 1, the internal state vector y progresses correctly through the one-hot encoded states.

Part 2. Modified One-Hot FSM

In the second part of the lab, the One-Hot FSM from Part 1 was modified to use a reset state encoded as all zeros, making it more compatible with FPGA hardware. Unlike the original design where $y(0)$ was set to '1' on reset, the modified version cleared all bits in the state vector y, implicitly representing state A as the all-zero condition. This approach leveraged the FPGA's built in clear lines, which reset flip-flops to zero.

The FSM logic remains mostly unchanged. When $y = "000000000"$, the FSM checks the input w to decide whether to start a sequence of 1s or 0s, transitioning to state B ($y(1)$) or F ($y(5)$) respectively. In normal operation, the FSM clears all bits in y each clock cycle and sets only the next active state, maintaining one-hot behavior. The output z is asserted when the FSM reaches state E ($y(4)$) or I ($y(8)$), signaling the detection of four consecutive 1s or 0s.

To support this logic, flip-flops were updated to use a synchronous active low reset, ensuring that each is cleared only on the rising edge of the clock. This change is reflected in the modified D flip-flop code, where the asynchronous reset condition used in Part 1:

```

--IF (Resetn = '0' and rising_edge(Clock)) THEN
  Q <= '0';

```

was replaced in Part 2 with a synchronous version:

```

IF rising_edge(Clock) THEN
  IF (Resetn = '0') THEN -- synchronous clear
    Q <= '0';
  ELSE

```

This reset style complements the FSM's updated behavior and ensures efficient mapping on FPGA hardware.

```

18   if rising_edge(clk) then
19       if reset_n = '0' then
20           y <= (others => '0'); -- Reset to state A (all 0s)
21       else
22           y <= (others => '0'); -- Clear previous state (simulate one-hot)
23       end if;
24       if y = "00000000" then
25           -- FSM is in reset state (A), start new sequence
26           if w = '1' then
27               y(1) <= '1'; -- A → B
28           else
29               y(5) <= '1'; -- A → F
30           end if;
31       else
32           -- FSM is active, proceed with transitions
33           y(1) <= w and y(0); -- A → B
34           y(2) <= w and y(1); -- B → C
35           y(3) <= w and y(2); -- C → D
36           y(4) <= w and y(3); -- D → E (z = 1)
37
38           y(5) <= (not w) and y(0); -- A → F
39           y(6) <= (not w) and y(5); -- F → G
40           y(7) <= (not w) and y(6); -- G → H
41           y(8) <= (not w) and y(7); -- H → I (z = 1)
42       end if;
43   end if;
44 end process;
45
46 -- Output is 1 in state E or I (detected 4 consecutive 1s or 0s)
47 z <= y(4) or y(8);
48 modified_onehot;
49 end
50

```

The testbench remains unchanged from Part 1, as the FSM's behavior is functionally the same. It applies an active low reset and drives the input w with sequences of four 0s, four 1s, random inputs, and another set of four 0s. The FSM output z goes high after each valid sequence, confirming correct operation.

The waveform generated by the simulation confirms that the FSM transitions correctly through the one-hot states and produces the expected output. The output z goes high at the appropriate points, and the state vector y changes in accordance with the input. This confirms that the modified FSM maintains functional correctness while using an all zero reset state.

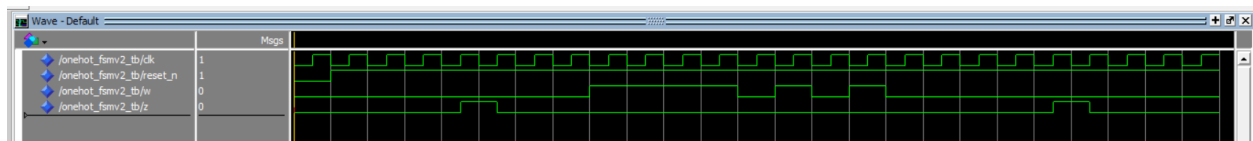


Figure 2. Modified FSM waveform showing correct z output after valid sequences.

Part 3. Two-process FSM

In this part of the lab, the FSM from Part 1 was reimplemented using a two-process VHDL design. Instead of manually deriving logic expressions for each flip-flop, the FSM was written using the two-process structure described in Figure 3 of the lab assignment. A CASE statement was used to define the next state, and a separate clocked process was used to update the current state. This made the design more structured and easier to scale.

The FSM states were defined using an enumerated type, and two signals were used to track the current and next states:

```

12 ARCHITECTURE behaviour OF two_process_FSM IS
13
14     TYPE State_type IS (A, B, C, D, E, F, G, H, I);
15     SIGNAL CS, FS : State_type; -- CS: current state, FS: next state
16     SIGNAL w, reset, clk, z : STD_LOGIC;
17

```

The clocked process updates the state on the rising edge of the clock or resets it to state A when the active low reset is asserted (this caused some anomaly that will be explained later):

```

28 -- Clocked process: updates current state on rising clock edge
29 PROCESS(cclk)
30 BEGIN
31   IF rising_edge(cclk) THEN
32     IF reset = '0' THEN
33       CS <= A;
34     ELSE
35       CS <= FS;
36     END IF;
37   END IF;
38 END PROCESS;
39

```

The next-state logic is described using a CASE statement. From state A, for example, the FSM moves to B if w = 0, or to F if w = 1, starting the detection of either 0s or 1s:

```

PROCESS(w, CS)
BEGIN
  CASE CS IS
    WHEN A =>
      IF w = '0' THEN FS <= B;
      ELSE FS <= F;
      END IF;
    -- (similar logic continues for other states)
  END CASE;
END PROCESS;

```

The output z is set high in states E and I, indicating that the FSM has detected four consecutive 0s or 1s:

```

87 -- Output logic: sets z high in states E and I
88 PROCESS(CS)
89 BEGIN
90   CASE CS IS
91     WHEN E =>
92       Z <= '1';
93     WHEN I =>
94       Z <= '1';
95     WHEN OTHERS =>
96       Z <= '0';
97   END CASE;
98 END PROCESS;
99 END behaviour;
100

```

As instructed in the lab manual, the RTL Viewer in Quartus was used to check the synthesized state diagram and compare it with the original FSM from the lab handout. Figure 3 shows the state diagram generated based on our VHDL code.

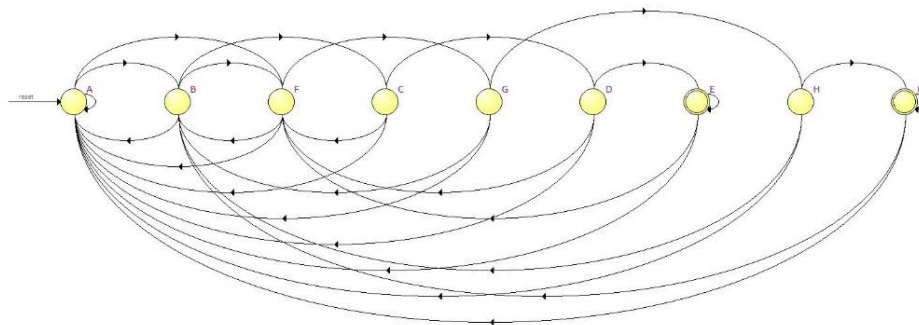


Figure 3. FSM of Part 3. Two-process FSM

Upon comparing this diagram with the original from the lab, the lab assistants were asked to confirm if the synthesized version was correct. They verified that it matched the expected behavior. However, it was noticed that each state appeared to have a transition back to state A, even though this was not explicitly defined in the CASE statements. This is likely because Quartus includes the reset logic in the FSM visualization. Since the FSM returns to state A when the synchronous reset is active, Quartus represents these transitions to A to reflect the reset path as part of the synthesized logic.

A testbench was created to simulate the clock, reset, and input w behavior. A 20 ns clock was generated, and an active low reset was applied to initialize the FSM. Sequences of four '0's and four '1's were used to trigger the output z, while a random alternating pattern ensured no false detection. A final set of four '0's confirmed that the FSM could detect valid sequences multiple times.

```

43 -- Stimulus process
44 stim_proc : process
45 begin
46     -- Apply reset for 1 cycle
47     reset_tb <= '0'; -- Active low
48     wait for 20 ns;
49     reset_tb <= '1';
50
51     -- Wait one cycle after reset
52     wait for 20 ns;
53
54     -- Apply four 0s (should trigger z)
55     w_tb <= '0'; wait for 80 ns;
56
57     -- Apply four 1s (should trigger z)
58     w_tb <= '1'; wait for 80 ns;
59
60     -- Apply random pattern (no z expected)
61     w_tb <= '0'; wait for 20 ns;
62     w_tb <= '1'; wait for 20 ns;
63     w_tb <= '0'; wait for 20 ns;
64     w_tb <= '1'; wait for 20 ns;
65
66     -- Apply another four 0s (should trigger z again) like previous parts
67     w_tb <= '0'; wait for 80 ns;
68
69     wait;
70 end process;

```

Figure 4. shows the simulation waveform of the FSM, where the output z correctly goes high after detecting four consecutive '0's or '1's, confirming the expected state transitions.

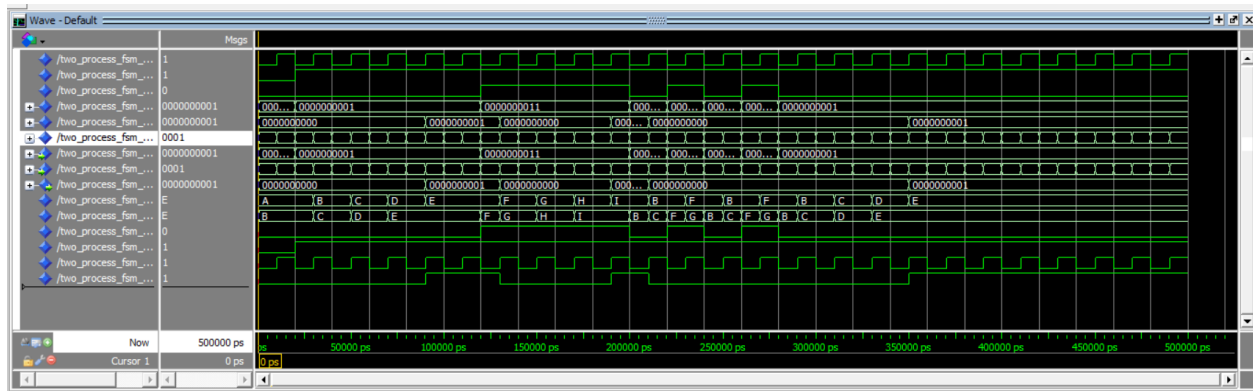


Figure 4. FSM simulation waveform showing input, output, and state transitions of part 3.

After changing the state encoding in Quartus from "Minimal Bits" to "One-Hot", the generated state codes were displayed in the Analysis and Synthesis section. As shown in Figure 5, each state is represented using a 9-bit one-hot format, where only one bit is high for each state. This matches the format shown in Table 2 of the lab manual. The reset state (A) is coded as all zeros, consistent with the modified one-hot approach used earlier in Part 2.

	Name	CS.I	CS.H	CS.G	CS.F	CS.E	CS.D	CS.C	CS.B	CS.A
1	CS.A	0	0	0	0	0	0	0	0	0
2	CS.B	0	0	0	0	0	0	0	1	1
3	CS.C	0	0	0	0	0	0	1	0	1
4	CS.D	0	0	0	0	0	1	0	0	1
5	CS.E	0	0	0	0	1	0	0	0	1
6	CS.F	0	0	0	1	0	0	0	0	1
7	CS.G	0	0	1	0	0	0	0	0	1
8	CS.H	0	1	0	0	0	0	0	0	1
9	CS.I	1	0	0	0	0	0	0	0	1

Name	State Code
	y ₈ y ₇ y ₆ y ₅ y ₄ y ₃ y ₂ y ₁ y ₀
A	00000000
B	00000011
C	00000101
D	00001001
E	00010001
F	00010001
G	00100001
H	01000001
I	10000001

Table 2 - Modified one-hot codes of the FSM.

Figure 5. One-hot state encoding generated by Quartus, matching the modified codes from Table 2 of the lab manual.

Part 4. "HELLO" FSM

The final part of the lab focuses on implementing a finite state machine that scrolls the word "HELLO" across six 7-segment displays on the DE1 board. Characters shift from right to left at one-second intervals using a pipeline of six 7-bit registers, each representing one display. The register structure follows the same design principles used in earlier labs. The FSM initially loads the characters "HELLO" into the pipeline and then loops continuously to maintain the scrolling effect. The following module defines the 7-bit register used in this pipeline.

```

16 begin
17   process (clk)
18   begin
19     if rising_edge(clk) then
20       if reset = '1' then
21         reg <= (others => '0');
22       elsif enable = '1' then
23         reg <= R;
24       end if;
25     end if;
26   end process;
27
28   Q <= reg;
29 end behavior;
30

```

The one_hz_counter module was used to generate a 1 Hz pulse from the 50 MHz clock by counting up to 49,999,999. Once the counter reaches this value, it resets and produces a one cycle pulse on the TC output, which is used to trigger the FSM and pipeline registers every second. The counter also includes a synchronous reset and an enable input. For simulation purposes, the counter value was temporarily reduced to speed up the scrolling and make the waveform easier to observe.


```

19 | process(cclk)
20 | begin
21 |   if rising_edge(cclk) then
22 |     if Clear = '1' then
23 |       count <= (others => '0');
24 |       pulse <= '0';
25 |     elsif Enable = '1' then
26 |       if count = 49999999 then
27 |         count <= (others => '0');
28 |         pulse <= '1'; -- start pulse
29 |       else
30 |         count <= count + 1;
31 |         pulse <= '0'; -- pulse low during count
32 |       end if;
33 |     else
34 |       pulse <= '0'; -- no counting
35 |     end if;
36 |   end if;
37 | end process;
38 |
39 | TC <= pulse;
40 |
41 | end behavior;
42 |

```

The Hello_FSM module controls the scrolling of the word “HELLO” across the six 7-segment displays on the DE1 board. It uses a state machine that steps through each character—H, E, L, L, O, and a blank space—and then enters a loop state to keep the word moving continuously. A 1 Hz pulse, generated by the one_hz_counter, controls the timing so that the characters shift left once per second. Each character is passed through a pipeline of six seven_bit_registers, with one register per display. After the initial sequence is loaded, the last register's output is fed back into the pipeline to maintain the scrolling effect. The 7-segment encodings for each character are stored as constants and selected based on the FSM state. Each register stage is then connected to one of the six HEX display outputs.

```

79 |
80 | -- FSM: Next-State Logic
81 | process(present_state)
82 | begin
83 |   case present_state is
84 |     when S_H    => next_state <= S_E;
85 |     when S_E    => next_state <= S_L1;
86 |     when S_L1   => next_state <= S_L2;
87 |     when S_L2   => next_state <= S_O;
88 |     when S_O    => next_state <= S_BLANK;
89 |     when S_BLANK => next_state <= S_LOOP;
90 |     when S_LOOP => next_state <= S_LOOP;
91 |     when others => next_state <= S_H;
92 |   end case;
93 | end process;
94 |
95 | -- FSM: Output Logic
96 | process(present_state)
97 | begin
98 |   case present_state is
99 |     when S_H    => char_input <= SEG_H;
100 |    when S_E    => char_input <= SEG_E;
101 |    when S_L1   => char_input <= SEG_L;
102 |    when S_L2   => char_input <= SEG_L;
103 |    when S_O    => char_input <= SEG_O;
104 |    when S_BLANK => char_input <= SEG_SPACE;
105 |    when S_LOOP => char_input <= stage5;
106 |    when others => char_input <= SEG_SPACE;
107 |   end case;
108 | end process;
109 |

```

To verify the functionality of the scrolling “HELLO” FSM, a testbench was created to simulate the clock, reset, and output behavior. A 50 MHz clock was generated using a simple process, and an active low reset was applied through KEY(0) for 100 ns before being released. The simulation was run long enough to observe the scrolling effect on the six 7-segment display outputs (HEX0 to HEX5). All outputs were connected to signals for waveform observation, confirming the FSM transitions and character shifting over time. The design was also successfully programmed onto the DE1 board, where the word “HELLO” scrolled smoothly across the displays, matching the expected behavior seen in simulation.

Conclusion

Through this lab, multiple FSM architectures were successfully implemented, simulated, and tested. Each part demonstrated a different aspect of FSM design, from simple sequence detection to real time visual output control. The final design, which scrolls the word “HELLO” across 7-segment displays, showed how FSMs can be used for coordinated timing and output sequencing. Overall, the lab provided hands on experience with both simulation and hardware validation of VHDL based FSMs.